

Week 4 · 개념 워크북: 값을 코드 밖으로 꺼내고, 리소스를 맵으로 관리합니다

참고

이 문서는 개념 파트입니다. 명령을 직접 입력하는 실습은 같은 `lecture/` 폴더의 `실습워크북` 을 따라가세요. `[개념]` 은 개념 설명, `[함정]` 은 미리 알고 피해 가는 것입니다.

지난주에 `terraform.tfstate` 를 S3로 옮겼습니다. 그 과정에서 버킷 이름을 직접 입력했고, `t3.micro` 와 `10.0.1.0/24` 는 코드에 직접 작성된 채 넘어왔습니다. 오늘 그 값들을 코드 밖으로 빼내고, 서버넷 블록 하나로 서버넷 두 개를 만듭니다.

이 문서를 덮을 때 세 가지를 말할 수 있어야 합니다. `variable` 과 `locals` 를 무엇으로 가르는지, `for_each` 가 리소스 주소에 무엇을 붙이는지, 리스트 가운데 원소를 지우면 왜 뒤가 전부 재생성되는지입니다.

체크포인트

week1·2·3에서 이미 설명한 것은 다시 풀지 않고 위치만 가리킵니다. VPC · 서버넷 · IGW · 라우트 테이블 · 시큐리티 그룹 · AMI · CIDR · 참조와 그래프는 week2 개념워크북 Part 0 · Part 1 · Part 2에 있고, backend · 원격 state · 잠금은 week3 개념워크북 Part 0에 있습니다.

오늘은 week3 개념워크북 19번이 Week4로 넘긴 표를 그대로 회수합니다.

week3에서 넘긴 것	오늘 어디서 받는가
<code>t3.micro</code> · <code>10.0.1.0/24</code> · <code>AES256</code> 이 코드에 직접 작성되어 있습니다. 하드코딩된 값을 <code>variables</code> 로 분리합니다	Part 1
접두어 하나로 되지 않고 이름 목록 자체가 여러 개라면 어떻게 쓸까요 (<code>for_each</code> vs <code>count</code>)	Part 4
<code>"\${var.project_name}-tfstate"</code> 같은 조립을 여러 파일에 반복했습니다. 조립식을 한 곳에 모으는 위치가 필요합니다 (<code>locals</code>)	Part 3
bootstrap의 output을 사람이 읽어 app에 옮겨 적었습니다. output 문법 자체를 다음 주에 정면으로 봅니다 (<code>output</code> , <code>sensitive</code>)	Part 2의 6번 · Part 5의 13번
backend는 변수를 못 받습니다. 그럼 나머지 값은 어디까지 변수로 뺄 수 있을까요	Part 6의 16번

오늘 다룰 것

참고

세션은 60분이고 그중 45분이 실습입니다. 그래서 이 문서는 대부분 연습용입니다. 라이브에서 다루는 것은 Part 3 · 4 · 6뿐이고 합쳐서 14분입니다. 나머지는 읽고 오세요. 읽지 않으면 실습 중에 왜 이렇게 하는지 모른 채 명령만 따라

입력하게 됩니다.

Part	섹션	무엇을 다루나	언제
Part 0	용어 사전	오늘 처음 나오는 말 18개	예습
Part 1	1 · 2 · 3	하드코딩이 남긴 자리와 타입	예습 · 라이브 3분
Part 2	4 · 5 · 6	값이 들어올 때 걸러내기	예습
Part 3	7 · 8	조립식을 한 곳에 모으기	라이브 3분
Part 4	9 · 10 · 11 · 12	리소스 하나를 N개로	라이브 6분
Part 5	13 · 14 · 15	여러 개가 된 것을 다루기	예습
Part 6	16 · 17 · 18	week3 자산 이어 쓰기, 비용과 정리	라이브 2분
Part 7	19	다음 주로 넘기는 문제	예습

오늘 나와야 하는 숫자

실습 중에 이 숫자와 다르면 멈추고 물어보세요.

지점	나와야 하는 값
count-demo <code>apply</code>	Apply complete! Resources: 6 added, 0 changed, 0 destroyed.
count-demo <code>plan</code> (bravo 삭제 후)	Plan: 1 to add, 0 to change, 3 to destroy.
app <code>plan</code>	Plan: 9 to add, 0 to change, 0 to destroy.
app <code>state list</code>	11줄 (리소스 9 + 데이터 소스 2)
S3 객체	week04/app/terraform.tfstate 1개
app <code>destroy</code>	Destroy complete! Resources: 9 destroyed.

9 와 11 과 6 이 각각 다른 뜻입니다. 9는 app 스택이 만드는 리소스 수, 11은 데이터 소스 두 줄이 더 붙은 `state list` 줄 수, 6은 count-demo가 만드는 `terraform_data` 개수입니다.

주의

week3에서는 9 가 `state list` 줄 수였습니다. 오늘은 같은 9 가 리소스 개수입니다. 숫자만 보고 넘기지 말고 어느 명령의 출력인지 같이 보세요.

Part 0. 용어 사전

오늘 처음 나오는 단어를 실습 전에 다 풀어둡니다. week1·2·3에서 정의한 용어는 넣지 않았습니다. 비유는 이 표 안에서만 쓰고, 본문에서는 왼쪽 열의 실제 이름으로 부릅니다.

용어	뜻	비유
타입 제약 (type constraint)	<code>variable</code> 블록의 <code>type</code> 인자에 적는, 이 변수가 받을 값의 형태	-
원시 타입 (primitive type)	더 쪼갤 수 없는 값 하나. <code>string</code> · <code>number</code> · <code>bool</code> 셋입니다	-
<code>list(...)</code>	순서가 있고 중복을 허용하는 값 묶음. 0부터 세는 번호로 꺼냅니다	줄 세운 대기표
<code>set(...)</code> / <code>toset()</code>	순서가 없고 중복이 없는 값 묶음. 번호로 꺼낼 수 없습니다. <code>toset()</code> 은 <code>list</code> 를 <code>set</code> 으로 바꾸는 함수이고 <code>for_each</code> 에 <code>list</code> 를 넣을 때 씁니다	이름표 바꾸니
<code>map(...)</code>	문자열 <code>key</code> 와 값의 쌍. <code>key</code> 로 꺼냅니다	번호 붙은 사물함
<code>object({...})</code>	필드 이름과 필드마다의 타입까지 미리 정해둔 묶음	칸이 정해진 신청서
<code>validation</code> 블록	변수에 들어온 값이 조건을 만족하는지 <code>plan</code> 전에 검사하는 블록. 인자 <code>condition</code> 이 <code>true</code> 여야 통과하고, <code>false</code> 면 <code>error_message</code> 가 그대로 화면에 나옵니다	-
<code>can()</code>	식을 실행해보고 오류 없이 끝나면 <code>true</code> 를 돌려주는 함수. 형식 검사에 씁니다	-
<code>sensitive</code>	값을 CLI 출력에서 <code>(sensitive value)</code> 로 가리는 표시. 저장까지 막지는 않습니다	영수증의 카드번호 별표
<code>locals</code> (지역값)	이 스택 안에서만 쓰는, 이름을 붙인 식. 바깥에서 값을 넣을 수 없습니다	-
<code>merge()</code>	맵 여러 개를 하나로 합치는 함수. 같은 <code>key</code> 가 겹치면 뒤에 적은 쪽의 값이 적용됩니다	나중에 붙인 라벨이 위에
메타 인자 (meta-argument)	리소스 타입이 무엇이든 <code>resource</code> · <code>data</code> · <code>module</code> 블록에 쓸 수 있는 인자. <code>count</code> · <code>for_each</code> · <code>depends_on</code> · <code>lifecycle</code> · <code>provider</code> · <code>output</code> 이나 <code>variable</code> 에는 쓰지 못합니다	-
<code>for_each</code> / <code>each.key</code> / <code>each.value</code>	블록 하나를 컬렉션의 원소 수만큼 복제하는 메타 인자. 복제가 도는 동안 지금 원소의 <code>key</code> 와 값을 <code>each</code> 로 가리킵니다	-
<code>count</code> / <code>count.index</code>	블록을 숫자만큼 복제하는 메타 인자와, 지금 몇 번째인지 알려주는 0부터 시작하는 번호	-

용어	뜻	비유
인스턴스 key (instance key)	복제된 리소스를 구분하는 꼬리표. <code>for_each</code> 는 <code>["a"]</code> , <code>count</code> 는 <code>[0]</code> . state 주소의 일부가 됩니다	-
<code>for</code> 표현식	컬렉션을 돌면서 새 컬렉션을 만드는 문법. <code>{ for k, v in ... : ... => ... }</code>	-
<code>terraform_data</code>	provider 없이 쓸 수 있는 내장 리소스. AWS 자격증명도 과금도 없어서 오늘 <code>count</code> 와 <code>for_each</code> 를 비교하는 샌드박스입니다	-
<code>precondition</code>	<code>lifecycle</code> 블록 안에 두는 검사. 이 리소스를 만들기 전에 조건이 참인지 plan 단계에서 확인합니다	-

Part 1. 하드코딩이 남긴 자리

[개념] 1. week3 코드에서 값이 직접 작성되어 있던 위치

week3의 `app/network.tf` 와 `compute.tf` 를 다시 보면 이런 값들이 리소스 블록 안에 직접 적혀 있었습니다.

값	어디에	오늘 어디로
<code>"10.0.0.0/16"</code>	<code>aws_vpc.main.cidr_block</code>	<code>var.vpc_cidr</code>
<code>"10.0.1.0/24"</code>	<code>aws_subnet.public.cidr_block</code>	<code>var.subnets["a"].cidr</code>
<code>names[0]</code>	<code>aws_subnet.public.availability_zone</code>	<code>var.subnets["a"].az</code>
<code>"\${var.project_name}-vpc"</code> 같은 조립	리소스마다 반복	<code>local.name_prefix</code>
태그 다섯 줄	<code>providers.tf</code> 의 <code>default_tags</code> 안에 직접	<code>local.common_tags</code>

값이 리소스 블록 안에 있으면 두 가지 문제가 생깁니다. 값을 바꾸려면 리소스 정의를 열어야 하고, 같은 리소스를 하나 더 만들려면 블록을 통째로 복사해야 합니다. 오늘 앞의 문제를 Part 1에서, 뒤의 문제를 Part 4에서 다룹니다.

[개념] 2. 타입: string 에서 map(object) 까지

1번에서 빼낼 값을 모두 찾았습니다. 이제 그 값을 받을 `variable` 블록을 만들 차례입니다.

블록을 열면 곧바로 문제가 발생합니다. `t3.micro` 는 값 하나로 충분하지만 서브넷 목록은 그렇지 않습니다. 서브넷 하나를 만들려면 CIDR 과 AZ 가 둘 다 필요하고, 서브넷이 두 개면 그 쌍이 두 벌 있어야 합니다. 받아야 하는 값의 구조는 이렇습니다.

```
a -> cidr 10.0.1.0/24   az ap-northeast-2a
c -> cidr 10.0.2.0/24   az ap-northeast-2c
```

이 모양을 Terraform 에 미리 정의하는 것이 타입입니다. "이 변수는 이렇게 생긴 값만 받는다"를 선언에 작성하면, tfvars 에 다른 모양을 입력했을 때 그 위치에서 검사에 걸립니다.

`type` 인자에 적을 수 있는 것은 세 갈래입니다.

갈래	무엇	예
원시 타입	<code>string</code> · <code>number</code> · <code>bool</code>	<code>type = string</code>
컬렉션 타입	<code>list(...)</code> · <code>set(...)</code> · <code>map(...)</code> . 원소 타입이 전부 같아야 합니다	<code>type = map(string)</code>
구조 타입	<code>object({...})</code> · <code>tuple([...])</code> . 필드마다 타입이 다를 수 있습니다	<code>type = object({ cidr = string, az = string })</code>

오늘 서브넷 목록에는 `map(object({ cidr = string, az = string }))` 를 사용합니다. 담아야 하는 값이 CIDR 과 AZ 둘이므로, `map(string)` 으로는 AZ 를 저장할 위치가 없습니다. 이 맵의 기본값에 `a` 와 `c` 를 key 로 두고, 이 두 글자가 Part 4에서 리소스 주소가 됩니다. 선언은 실습워크북 A-4에서 직접 작성합니다.

`list` 와 `set` 은 오늘 변수로 쓰지 않습니다. `for_each` 에 `list` 를 넣을 때 `toiset()` 으로 감싸는 12번에서만 나옵니다.

[함정] 3. `type` 을 안 적으면 `any` 가 됩니다

`type` 은 생략할 수 있습니다. 생략하면 Terraform이 어떤 값이든 받습니다.

```
variable "subnets" {  
  default = { a = "10.0.1.0/24" } # type 없음  
}
```

이렇게 두면 tfvars 에서 문자열을 넣든 숫자를 넣든 통과하고, 오류가 이 변수를 쓰는 리소스에서 납니다. 오류 메시지는 변수 이름 대신 리소스 인자를 가리키므로 원인을 찾기가 어려워집니다.

`type` 을 작성하는 것은 문서화이기도 합니다. 무엇을 입력해야 하는지가 선언에 명시되어 있으면 `example.tfvars` 를 찾아볼 필요가 없습니다.

Part 2. 값이 들어올 때 걸러내기

Part 1에서 타입을 정했습니다. 그런데 타입은 값의 생김새만 봅니다. 안에 적힌 내용이 맞는지는 보지 않습니다.

`type = string` 이라고 적어두면 아래 셋이 전부 통과합니다.

<code>"10.0.1.0/24"</code>	맞는 값
<code>"10.0.1.0/240"</code>	/240 이라는 대역은 없습니다
<code>"10.0.1"</code>	CIDR 표기가 아닙니다

`instance_type` 도 같습니다. 검사를 따로 걸어두지 않으면 `m5.24xlarge` 도 문자열이라 `string` 을 그대로 지나가고, 요금이 전혀 다른 인스턴스가 만들어집니다.

그래서 형태를 지나온 값을 한 겹 더 거릅니다. 거르는 문법을 4번에서 보고, 그 문법을 쓸 때 걸리는 함정을 5번에서 봅니다. 값이 맞더라도 화면에 띄우면 안 되는 경우는 6번입니다.

[개념] 4. `validation` 은 `plan` 이 시작되기 전에 막습니다

week2 실습워크북 A-2에서 잘못된 값을 넣었다가 오류를 본 적이 있습니다. 그 오류를 내던 쪽을 오늘 직접 씁니다.

`validation` 블록은 `condition` 과 `error_message` 두 인자를 받고, `condition` 이 `false` 면 `error_message` 가 그대로 화면에 나오면서 멈춥니다.

```
variable "instance_type" {
  type    = string
  default = "t3.micro"

  validation {
    condition      = contains(["t3.micro", "t2.micro"], var.instance_type)
    error_message = "이번 실습은 t3.micro 또는 t2.micro 만 허용합니다 (과금 방지)."
```

검사가 실행되는 시점이 중요합니다. AWS에 요청을 보내기 전, `plan` 이 리소스를 계산하기 전에 멈추므로 아무것도 생성되지 않습니다. 검사가 없으면 잘못된 값이 `apply` 진행 중에 발견되어 VPC와 IGW는 생성되지만 서브넷에서 실패합니다. 이러한 상태는 수동으로 정리하거나 다시 `apply` 해야 합니다.

Terraform 1.9부터 `condition` 이 다른 변수를 참조할 수 있습니다. 오늘 `primary_subnet_key` 에 `condition = contains(keys(var.subnets), var.primary_subnet_key)` 를 걸어서 그 기능을 씁니다. `versions.tf` 의 `required_version = ">= 1.9.0"` 이 이것 때문에 필요하니 낮추지 마세요.

일부러 틀린 값을 넣으면 `plan` 이 시작되기 전에 `Error: Invalid value for variable` 이 나고, 마지막 줄에 우리가 적은 `error_message` 가 그대로 찍힙니다. 출력 전문은 실습워크북 A-6에 있습니다.

[함정] 5. `condition` 에 `can()` 을 씌우는 이유

CIDR 형식을 검사하고 싶을 때 이렇게 쓰고 싶어집니다.

```
condition = cidrnetmask(var.vpc_cidr) != ""
```

값이 CIDR 형식일 때는 통과하지만, 형식이 아니면 `cidrnetmask()` 자체가 오류를 내면서 실패합니다. `condition` 이 `false` 를 돌려주는 대신 식 평가가 실패하므로, 화면에는 우리가 적은 `error_message` 대신 함수의 오류가 나옵니다.

`can()` 은 식을 실행해보고 오류 없이 끝났는지만 알려줍니다.

```
condition = can(cidrnetmask(var.vpc_cidr))
```

정규식보다 `can(cidrnetmask(...))` 가 나은 이유도 같습니다. `"999.1.1.1"` 은 정규식 `\d+\.\d+\.\d+\.\d+` 를 통과하지만 `cidrnetmask()` 는 통과하지 못합니다.

맵의 모든 값에 같은 검사를 걸 때는 `for` 표현식과 `alltrue()` 를 함께 씁니다. `for` 표현식은 13번에서 다시 봅니다.

```
condition = alltrue([for s in values(var.subnets) : can(cidrnetmask(s.cidr))])
```

[개념] 6. `sensitive` 는 화면을 가립니다. `state` 는 평문입니다

4번과 5번은 잘못된 값을 들여보내지 않는 이야기였습니다. 6번은 값이 맞게 들어온 다음의 이야기입니다.

`my_ip` 에는 본인 공인 IP 를 적습니다. 값 자체는 올바릅니다. 문제는 이 값이 시큐리티 그룹의 `cidr_blocks` 로 들어간다는 것입니다. 아무 표시도 하지 않으면 `plan` 출력에 그대로 찍히고, 그 화면을 캡처해 제출하면 어느 IP 에 22번 포트를 열어두었는지가 공개 리포에 남습니다.

값은 맞는데 보이면 안 되는 경우이고, 그것을 다루는 표시가 `sensitive` 입니다.

week3에서 `state` 가 평문 JSON이라는 것을 봤습니다. `sensitive` 는 그 사실을 바꾸지 않습니다. 바꾸는 것은 CLI 출력입니다.

오늘 `my_ip` 에 `sensitive = true` 를 붙입니다. 본인 공인 IP는 이 스터디에서 계속 가리라고 해온 값이고, 실습 화면을 캡처해 제출하기 때문입니다.

```
variable "my_ip" {
  type      = string
  sensitive = true
}
```

표시하면 `plan` 출력에서 이 값을 사용하는 부분이 전체적으로 숨겨집니다. 시큐리티 그룹의 `ingress` 블록 전체가 이렇게 표시됩니다.

```
+ ingress          = (sensitive value)
```

그리고 이 값을 참조하는 output 에도 `sensitive` 가 있어야 합니다. 빼면

`Error: Output refers to sensitive values` 로 `plan` 이 멈춥니다. 출력 전문은 실습워크북 B-3에 있습니다.

값이 정말 필요하다면 `terraform output -raw ssh_source_cidr` 로 꺼낼 수 있고, `terraform state pull` 로 `state` 를 열면 그 안에 평문으로 들어 있습니다. `sensitive` 가 막는 것은 화면이고, 저장 쪽은 week3에서 건 버킷 암호화와 퍼블릭 접근 차단이 맞습니다.

Part 3. 조립식을 한 곳에 모으기

Part 2까지 오면서 값을 밖에서 받고 걸러내는 것까지 했습니다. 이제 받은 값을 코드 안에서 쓰는 쪽을 봅니다.

week3 코드에는 이런 줄이 파일마다 흩어져 있었습니다.

```
tags = { Name = "${var.project_name}-vpc" }
tags = { Name = "${var.project_name}-igw" }
tags = { Name = "${var.project_name}-rt-public" }
```

`project_name` 은 사람이 tfvars 에 입력하는 값입니다. 그런데 여기에 `-vpc` 를 추가하는 것은 사람이 결정하는 것이 아닙니다. `project_name` 이 정해지면 자동으로 따라옵니다. 접두어 규칙을 변경하려면 이 줄 전체를 찾아 수정해야 하는데, 하나를 놓쳐도 오류가 발생하지 않고 리소스 이름만 경고 없이 불일치하게 됩니다.

사람이 정하는 값과 그로부터 파생되는 값을 같은 위치에 두면 tfvars 에 무엇을 입력해야 하는지도 불명확해집니다. 그래서 파생되는 값은 별도로 모아둡니다.

1번의 표에서 `local.name_prefix` 와 나란히 예고한 `local.common_tags` 도 같은 이유로 여기에 들어옵니다. 리소스마다 반복해 적던 태그 다섯 줄을 한 곳에 모으는 것이고, 그러려면 맵 둘을 합쳐야 해서 8번에서 `merge()` 를 봅니다.

[개념] 7. `locals` : 계산한 값에 이름을 붙이는 위치

`variable` 과 `locals` 를 구분하는 기준은 하나입니다. 외부에서 값을 입력할 수 있는가입니다.

	variable	locals
값이 오는 곳	tfvars · <code>-var</code> · 환경변수 · <code>default</code>	다른 값으로부터 계산
바깥에서 바꾸기	됩니다	안 됩니다
<code>validation</code>	붙일 수 있습니다	없습니다
쓰는 법	<code>var.이름</code>	<code>local.이름</code>

그래서 `project_name` 은 `variable` 이고, 이것으로 조립한 `name_prefix` 는 `locals` 입니다. 사람이 정하는 값과 그로부터 파생되는 값을 분리하면 tfvars 에 입력할 내용이 명확해집니다.

week3에서는 `"${var.project_name}-vpc"`, `"${var.project_name}-igw"` 같은 조립을 파일마다 반복해서 작성했습니다. 접두어 규칙이 변경되면 모든 위치를 찾아 수정해야 했습니다. 오늘은 `local.name_prefix` 하나를 통해 처리합니다.

[개념] 8. `merge()` : 뒤에 적은 맵이 우선합니다

`merge()` 는 맵 여러 개를 하나로 합칩니다. 같은 key 가 여러 맵에 있으면 뒤에 적은 쪽 값이 남습니다.

```
locals {
  common_tags = merge(
    var.extra_tags,
    {
      Project   = var.project_name
      Study     = "boaz-terraform-26"
      Week      = "4"
      ManagedBy = "terraform"
      Purpose   = "workload"
    }
  )
}
```

```

    }
  )
}

```

순서가 안전 장치입니다. `var.extra_tags` 를 앞에 두었으므로, 누군가 `tfvars` 에 `extra_tags = { Purpose = "test" }` 를 입력해도 뒤의 리터럴 맵이 우선되어 `Purpose` 는 `workload` 로 유지됩니다. `Purpose` 태그는 `scripts/check-leftover.sh` 가 "오늘 제거해야 하는 리소스"와 "week3이 남겨둔 state 저장소"를 구분하는 기준이므로, 순서를 바꾸면 정리 확인이 필수에서 선택으로 변경됩니다.

합쳐진 맵은 `providers.tf` 의 `default_tags` 로 들어갑니다. 태그가 붙는 층이 둘로 나뉘어서, 모든 리소스에 공통으로 붙는 것은 `default_tags` 가 처리하고 리소스마다 달라지는 `Name` 만 각 리소스의 `tags` 에 적습니다. 리소스 블록의 태그가 한 줄로 짧아집니다.

```
tags = { Name = "${local.name_prefix}-public-${each.key}" }
```

Part 4. 리소스 하나를 N개로

Part 1의 1번에서 하드코딩이 두 가지를 막는다고 했습니다. 값을 바꾸려면 리소스 정의를 열어야 하는 것, 그리고 같은 리소스를 하나 더 만들려면 블록을 통째로 복사해야 하는 것입니다. 앞의 것을 Part 3까지 풀었고 여기서 뒤의 것을 풀니다.

서브넷을 두 개로 만드는 가장 단순한 방법은 블록을 복사하는 것입니다.

```

resource "aws_subnet" "public_a" { ... }
resource "aws_subnet" "public_c" { ... }

```

이러면 서브넷이 다섯 개일 때 블록이 다섯 개가 되고, 공통으로 고칠 것이 생기면 다섯 군데를 고쳐야 합니다. 대신 블록 하나에 "이 블록을 몇 벌 만들어라"를 붙이는 방법이 있습니다. 그런 인자를 메타 인자라고 부르고, 리소스 타입이 무엇이든 `resource` · `data` · `module` 블록에 붙일 수 있습니다. `output` 이나 `variable` 에는 붙지 않습니다.

복제용 메타 인자는 `for_each` 와 `count` 둘입니다. AWS 리소스에 쓰는 것은 `for_each` 하나인데도 둘을 나란히 보는 이유가 있습니다. 코드만 보면 둘이 거의 같게 생겨서, 하나를 골라야 하는 근거가 문법이 아닌 곳에 있기 때문입니다. 그 근거를 11번에서 보고, 고른 다음 실제로 값을 넣을 때 걸리는 것을 12번에서 봅니다. `count` 쪽은 실습워크북 A-2의 샌드박스에서 직접 돌려봅니다.

[개념] 9. `for_each` : 리소스 주소에 `key` 가 붙습니다

`for_each` 는 메타 인자입니다. 리소스 블록 하나를 컬렉션의 원소 수만큼 복제합니다.

```

resource "aws_subnet" "public" {
  for_each = var.subnets

  vpc_id      = aws_vpc.main.id
  cidr_block  = each.value.cidr
}

```

```
availability_zone = each.value.az

tags = { Name = "${local.name_prefix}-public-${each.key}" }
}
```

블록은 하나인데 리소스는 두 개가 생성됩니다. 복제가 진행되는 동안 `each.key` 가 map의 key를, `each.value` 가 해당 값을 가리킵니다. 현재 `each.value` 는 `{ cidr = "...", az = "..." }` 객체이므로 `each.value.cidr` 처럼 필드를 추출하여 사용합니다.

생성된 리소스의 주소에는 key 가 붙고, 이 주소가 state 에 그대로 저장됩니다. `terraform state list` 에도 이 형식으로 표시됩니다.

```
aws_subnet.public["a"]
aws_subnet.public["c"]
```

[개념] 10. `count` : 리소스 주소에 번호가 붙습니다

같은 작업을 `count` 로도 수행할 수 있습니다. 이 방식은 개수를 숫자로 받고, `count.index` 로 현재 몇 번째인지 알려줍니다.

```
resource "terraform_data" "by_count" {
  count = length(var.names)

  triggers_replace = var.names[count.index]
  input            = var.names[count.index]
}
```

주소에는 0부터 시작하는 인덱스 번호가 붙습니다.

```
terraform_data.by_count[0]
terraform_data.by_count[1]
terraform_data.by_count[2]
```

여기까지는 둘이 같아 보입니다. 차이는 목록이 바뀔 때 드러납니다.

[함정] 11. 목록의 중간 원소를 제거하면 (오늘의 핵심)

실습 A-2에서 직접 수행할 내용을 미리 확인합니다. `["alpha", "bravo", "charlie"]` 를 apply 한 뒤 중간의 `"bravo"` 를 삭제하고 `plan` 을 실행하면 다음과 같이 나옵니다.

```
# terraform_data.by_count[1] must be replaced
-/+ resource "terraform_data" "by_count" {
  ~ id            = "126a2864-24da-76a5-f1bb-d4f94985d892" -> (known after apply)
  ~ input         = "bravo" -> "charlie"
```

```

~ output          = "bravo" -> (known after apply)
~ triggers_replace = "bravo" -> "charlie"
}

```

```

# terraform_data.by_count[2] will be destroyed
# (because index [2] is out of range for count)

```

```

# terraform_data.by_for_each["bravo"] will be destroyed
# (because key ["bravo"] is not in for_each map)

```

Plan: 1 to add, 0 to change, 3 to destroy.

`count` 쪽을 보면 무엇이 발생하는지 알 수 있습니다. "charlie" 는 값이 전혀 변경되지 않았는데, 목록에서 두 번째 위치로 이동하면서 [1] 이 되었습니다. [1] 에 원래 있던 것은 "bravo" 였습니다. state 는 리소스를 주소로 저장하므로 Terraform 입장에서는 [1] 의 내용이 "bravo" 에서 "charlie" 로 변경된 것이고, 따라서 교체(replacement)가 필요합니다. [2] 는 범위를 벗어나 삭제됩니다.

`for_each` 쪽은 ["bravo"] 하나만 제거되고 ["alpha"] 와 ["charlie"] 는 계획에 표시되지도 않습니다.

숫자를 분해하면 이렇습니다.

항목	add	destroy	이유
by_count[0]			alpha 그대로. 계획에 안 나옴
by_count[1]	1	1	교체 한 건이 add 와 destroy 양쪽에 1씩
by_count[2]		1	count 범위 밖
by_for_each["alpha"]			그대로
by_for_each["bravo"]		1	key 가 map 에 없음
by_for_each["charlie"]			그대로
합계	1	3	

plan 출력에 `# forces replacement` 표시는 나타나지 않고 `must be replaced` 만 표시됩니다. 교체가 필요한 이유는 `~ triggers_replace = "bravo" -> "charlie"` 줄을 확인하여 판단해야 합니다.

여기서 `terraform_data` 는 값이 변경되면 교체되도록 의도적으로 만든 것이고, 실제 AWS 리소스에서도 같은 동작이 발생합니다. 서버넷의 CIDR 이나 AZ 는 변경 불가능한 속성이라, 이 값이 변경되면 서버넷이 삭제되고 새로 생성됩니다. 서버넷 내에 인스턴스가 있으면 그 인스턴스도 함께 교체됩니다. 교체의 개념은 week2 개념워크북 12번에 설명되어 있으며, 목록이 바뀔 수 있는 대상에 `count` 를 사용하지 않는 이유가 바로 이것입니다.

[함정] 12. `for_each` 가 받을 수 있는 것은 map 과 set 뿐입니다

list 를 직접 전달하면 다음의 오류가 발생합니다.

Error: Invalid for_each argument

list를 사용하려면 `toiset()`으로 감싸야 합니다. set으로 변환하면 값 자체가 key가 되어 `each.key`와 `each.value`가 동일합니다. count-demo의 `terraform_data.by_for_each`가 그 형태입니다.

또 하나 있습니다. `for_each`의 key는 plan 시점에 값이 결정되어 있어야 합니다. 아직 생성되지 않은 리소스의 속성을 key로 사용하면 다음의 오류가 발생합니다.

The "for_each" value depends on resource attributes that cannot be determined until apply.

key는 변수나 상수로만 구성하세요. 값 쪽에는 리소스 속성을 사용할 수 있습니다. 오늘 `aws_route_table_association`에 `for_each = aws_subnet.public`을 적용하는 부분이 그 예입니다(실습 B-2).

`aws_subnet.public`은 `for_each`로 생성한 리소스이므로 그 자체가 맵입니다. 이 맵을 다시 `for_each`에 전달하면 key는 그대로 "a" · "c"로 유지되고, `each.value`는 서브넷 객체가 됩니다. 서브넷의 ID는 apply 이후에 결정되지만 key는 현시점에 알 수 있으므로 문제가 없습니다.

`var.subnets`를 다시 순회해도 결과는 같습니다. 리소스 쪽을 순회하면 서브넷을 먼저 만들라는 참조가 자연스럽게 생기고, 두 맵의 key가 어긋날 일이 없습니다.

Part 5. 여러 개가 된 것을 다루기

`for_each`로 서브넷이 두 개 되었습니다. 리소스가 하나였을 때는 없던 일이 여기서 생깁니다.

week3까지는 `aws_subnet.public.id`라고 쓰면 그 서브넷 하나를 가리켰습니다. 지금은 `aws_subnet.public`이 서브넷 하나가 아니라 맵입니다. 그래서 이 뒤에 `.id`를 붙일 수 없고, 세 가지를 새로 정해야 합니다. 두 개를 한꺼번에 output으로 꺼내는 방법, EC2를 둘 중 어디에 놓을지 지목하는 방법, 그리고 직접 적은 AZ가 이 계정에 실제로 있는지 확인하는 방법입니다.

[개념] 13. for 표현식

`for` 표현식은 컬렉션을 돌면서 새 컬렉션을 만듭니다. 이름이 비슷한 `for_each`는 리소스를 복제하는 메타 인자이고, `for` 표현식은 값을 가공하는 문법입니다.

맵을 만들 때는 중괄호를 쓰고 `=>`로 key와 값을 잇습니다. 오늘 `subnet_ids` output에 쓰는 식이

`{ for k, s in aws_subnet.public : k => s.id }`이고, 결과가 이렇게 나옵니다.

```
subnet_ids = {  
  "a" = "subnet-0abc..."  
  "c" = "subnet-0def..."  
}
```

새 값 자리에 객체를 넣으면 여러 필드를 함께 낼 수 있습니다. `outputs.tf` 의 `subnet_detail` 이 그 형태이고, key 마다 `az` 와 `cidr` 을 묶어서 냅니다.

대괄호를 쓰면 리스트가 나옵니다. 5번에서 본

`[for s in values(var.subnets) : can(cidrnetmask(s.cidr))]` 가 그 형태입니다. `values()` 는 맵에서 값만 꺼내는 함수이고, `keys()` 는 key 만 꺼냅니다.

[개념] 14. 여러 개 중 하나 고르기

서브넷이 두 개가 되면 EC2를 어디에 놓을지 골라야 합니다. 맵이므로 key 로 지목합니다.

```
subnet_id = aws_subnet.public[var.primary_subnet_key].id
```

`values(aws_subnet.public)[0]` 으로 쓰지 않는 이유가 오늘 주제와 같습니다. `values()` 의 순서는 맵 key 정렬을 따르므로, 나중에 key `"b"` 를 추가하면 `[0]` 이 가리키는 서브넷이 경고 없이 바뀝니다. 그러면 인스턴스가 통째로 재생성됩니다. 11번에서 본 것과 같은 사고가 참조 쪽에서 일어나는 것입니다.

`var.primary_subnet_key` 에 없는 key 를 적으면 `Invalid index` 라는 짧은 오류가 나옵니다. 그 오류는 어느 key 가 가능한지 알려주지 않습니다. 그래서 4번의 교차 참조 `validation` 으로 먼저 잡습니다.

셸에서 이 주소를 다룰 때는 `terraform state show 'aws_subnet.public["a"]'` 처럼 작은따옴표로 감싸야 합니다. 감싸지 않으면 셸이 대괄호와 큰따옴표를 먼저 해석합니다.

[함정] 15. AZ 를 다시 직접 적는 이유

week3에서는 `data.aws_availability_zones.available.names[0]` 으로 AZ 를 AWS에 물어봤습니다. AZ 이름은 계정마다 물리 AZ에 다르게 매핑되므로 하드코딩하지 말라고 했습니다. 그런데 오늘 `var.subnets` 의 값에 `az = "ap-northeast-2a"` 를 직접 적습니다.

서브넷마다 다른 AZ 를 배정해야 하기 때문입니다. 데이터 소스가 주는 것은 목록이고, 어느 서브넷을 어느 AZ에 놓을지는 사람이 정하는 결정이라 변수로 올립니다.

직접 적었으니 그 값이 맞는지 검사합니다. `lifecycle` 블록의 `precondition` 이 그 자리입니다.

```
lifecycle {
  precondition {
    condition      = contains(data.aws_availability_zones.available.names, each.value.az)
    error_message = "subnets[\"${each.key}\"].az 가 이 계정에서 쓸 수 없는 AZ입니다: ${each.value.az}"
  }
}
```

`validation` 으로 못 하는 이유는 변수의 `validation` 이 데이터 소스를 참조할 수 없기 때문입니다. `precondition` 은 리소스 안에 있어서 데이터 소스를 볼 수 있고, `each` 도 쓸 수 있습니다.

실제로 없는 AZ 를 넣으면 plan 에서 멈춥니다.

Error: Resource precondition failed

```
on network.tf line 56, in resource "aws_subnet" "public":
56:         condition      = contains(data.aws_availability_zones.available.names, each.value.az)
    |_____
    | data.aws_availability_zones.available.names is list of string with 4 elements
    | each.value.az is "ap-northeast-2z"
```

subnets["z"].az 가 이 계정에서 쓸 수 없는 AZ입니다: ap-northeast-2z

주의

통과한 인스턴스의 plan 이 먼저 찍히고 그 아래에 오류가 붙습니다. 위쪽의 **Plan:** 줄만 보고 성공했다고 넘기지 마세요.

서울 리전에서 쓸 수 있는 AZ 는 `ap-northeast-2a` · `ap-northeast-2b` · `ap-northeast-2c` · `ap-northeast-2d` 네 개입니다.

Part 6. week3 자산을 이어 쓰기, 비용과 정리

오늘 값을 거의 다 변수로 뺐습니다. 그런데 `backend.tf` 의 버킷 이름은 지난주와 똑같이 또 직접 적습니다.

여기까지 배운 것을 그대로 적용하면 `bucket = var.state_bucket` 이라고 쓸 수 있을 것 같습니다. 그렇게 쓰면 `init` 이 실패합니다. 변수로 뺄 수 있는 값과 뺄 수 없는 값의 경계가 여기서 드러나고, 그 경계가 Part 7에서 다음 주로 넘기는 문제가 됩니다.

리소스가 늘었으니 비용과 정리 절차도 여기서 다시 확인합니다.

[개념] 16. backend 는 여전히 변수를 못 받습니다

오늘 state 저장소를 새로 만들지 않습니다. week3의 bootstrap 스택이 만든 S3 버킷과 DynamoDB 테이블을 그대로 쓰고, `backend.tf` 에서 달라지는 것은 `key` 한 줄입니다. 오늘 값은 `week04/app/terraform.tfstate` 입니다. 같은 버킷 안에서 `key` 가 다르면 서로 다른 state 라서, `week03/app/terraform.tfstate` 는 그 자리에 그대로 남습니다.

버킷 이름을 또 직접 적는 이유는 week3 개념워크북 4번에서 본 것과 같습니다. `backend` 블록 안에서는 변수를 참조할 수 없습니다. `bucket = var.state_bucket` 이라고 적으면 `init` 이 이렇게 멈춥니다.

```
Initializing the backend...
```

```
Error: Variables not allowed
```

```
on backend.tf line 3, in terraform:
3:     bucket = var.state_bucket
```

Variables may not be used here.

`default` 가 지정되어 있어서 값이 이미 정해진 변수라도 같은 오류가 납니다. 값이 준비됐는지의 문제가 아니라, 이 자리에서 변수 참조 자체가 허용되지 않습니다. `init` 이 backend 를 가장 먼저 초기화한다는 것도 사실이지만 오류를 내는 쪽은 문법입니다. 그러니 `default` 를 주거나 `tfvars` 에 적어도 우회할 수 없습니다.

변수로 뺄 수 있는 값과 뺄 수 없는 값의 경계가 여기입니다.

주의

`key` 를 `week03/app/terraform.tfstate` 로 두면 week4 스택이 week3의 state 를 인수합니다. week3 리소스를 이미 지웠다면 첫 plan 이 `9 to add` 가 아니라 이상한 숫자로 나오고, 지우지 않았다면 대량의 삭제와 교체가 계획에 포함됩니다.

backend 블록에는 변수도 검사도 넣을 수 없어서 코드로 막을 방법이 없습니다. 첫 `plan` 이 `Plan: 9 to add, 0 to change, 0 to destroy.` 가 아니면 즉시 멈추세요.

week3의 버킷이 없으면 `init` 이 이렇게 실패합니다. week3을 하지 않았거나, 버킷을 지웠거나, `project_name` 을 다르게 적었을 때 모두 같은 오류가 납니다.

Error: Failed to get existing workspaces: S3 bucket "boaz26-w3-kdh1834-tfstate" does not exist.

복구는 week3 리포의 `bootstrap` 스택을 다시 apply 하는 것 하나뿐이고, 그때 `project_name` 은 week3에서 쓴 값과 글자 하나까지 같아야 합니다. `init` 중에 뜨는 `dynamodb_table deprecated` 경고는 week3에서 본 그 경고이고 정상입니다.

[개념] 17. 이번 주 비용: 리소스가 늘어도 청구서는 그대로입니다

서브넷이 1개에서 2개가 되고 라우트 테이블 연결도 2개가 되었지만, 비용은 week3과 같습니다.

항목	개수	시간당	730시간(한 달)
EC2 <code>t3.micro</code> (서울)	1	\$0.0130	\$9.49
퍼블릭 IPv4 주소	1	\$0.0050	\$3.65
EBS <code>gp3</code> 루트 8GiB	1	\$0.0010	\$0.73
서브넷	2	\$0	\$0
라우트 테이블 연결	2	\$0	\$0
VPC · IGW · 라우트 테이블 · 시큐리티 그룹	각 1	\$0	\$0
합계		\$0.0190	\$13.87

week3에서 만든 S3와 DynamoDB는 한 달 \$0.01 미만입니다. 오늘은 그 버킷에 객체 하나를 더 얹을 뿐이라 이 값도 그대로입니다.

상황	총 비용
3시간 실습하고 바로 destroy	약 \$0.06
destroy 를 잊고 한 달 방치	약 \$13.87

돈을 쓰는 것은 리소스 개수가 아니라 어떤 리소스인지입니다. 무료인 것은 두 배가 되어도 무료이고, 과금되는 것은 한 대만 늘어도 그만큼 늘어납니다.

비용·보안 경고

for_each

를 배운 직후에 EC2에 그대로 붙여보는 실수가 잦습니다. 맵이 2개면 시간당 \$0.0380, 3개면 \$0.0570이 됩니다.

compute.tf

에 이유를 주석으로 적어 두었습니다.

NAT Gateway 와 Elastic IP 도 만들지 마세요. NAT Gateway 는 시간당 과금과 데이터 처리 과금이 있고 프리티어가 없습니다.

stop 은 destroy 가 아닙니다. 인스턴스를 멈추면 EC2와 퍼블릭 IPv4 요금은 멈추지만 EBS 8GiB의 월 \$0.73은 계속 나갑니다.

[개념] 18. destroy 순서와 잔존 점검

지우는 것은 오늘 만든 것뿐입니다. week3의 S3와 DynamoDB는 7주차까지 남기며, 반대로 하면 state 저장소가 먼저 사라져서 오늘 만든 것을 지울 수단이 없어집니다. count-demo 도 지웁니다. 과금은 없지만 로컬 state 파일이 남아 제출물에 섞입니다.

확인은 두 겹으로 합니다.

terraform state list

가 비었다는 것은 Terraform이 더 이상 관리하지 않는다는 뜻일 뿐입니다. 콘솔에서 직접 만든 것이나 destroy 가 중간에 실패해 남은 것은 여기에 나타나지 않습니다.

계정 쪽 확인에는

Purpose

 태그가 쓰입니다.

Purpose	무엇	오늘 세션 끝에
workload	오늘 만든 VPC · 서브넷 · EC2	0개여야 합니다
terraform-state	week3의 S3 · DynamoDB	남아 있어야 정상입니다

scripts/check-leftover.sh

가 이 구분대로 조회합니다. 실습 C-3에서 실행합니다.

Part 7. 다음 주로 넘기는 문제

[개념] 19. 세 문장으로 줄이면

1. 값을 코드 밖으로 꺼내는 것은 `variable` 이고, 꺼낸 값에서 계산하는 것은 `locals` 입니다. 나누는 기준은 바깥에서 값을 넣을 수 있는가입니다.
2. `for_each` 는 리소스 주소에 `key` 를 붙이고, `count` 는 번호를 붙입니다. 목록이 바뀔 수 있으면 번호는 밀리고, 밀린 주소는 교체가 됩니다.
3. `validation` 은 잘못된 값이 `apply` 한복판까지 내려가지 못하게 세웁니다. 검사가 없으면 절반만 만들어진 상태에서 멈춥니다.

그리고 오늘 만든 것들이 다음 주의 문제를 그대로 만들어냅니다.

오늘 본 것	Week5에서 문제가 되는 지점
<code>variables.tf</code> 하나에 변수 8개가 모였습니다	이 묶음을 다른 폴더에서도 쓰려면 파일을 복사해야 합니다. 복사본은 곧 원본과 어긋납니다 (모듈의 input)
<code>outputs.tf</code> 에서 <code>subnet_ids</code> 를 맵으로 꺼냈습니다	지금은 사람이 화면으로 읽습니다. 코드가 코드에게 넘기려면 받는 쪽이 필요합니다 (모듈의 output)
<code>for_each</code> 로 서브넷을 2개로 늘렸습니다	VPC와 서브넷과 라우팅을 묶어서 한 세트씩 여러 벌 만들려면 <code>for_each</code> 를 어디에 걸까요 (<code>module</code> 블록의 <code>for_each</code>)
<code>local.common_tags</code> 를 이 폴더 안에서만 씁니다	<code>locals</code> 는 폴더 경계를 넘지 않습니다. 넘기려면 변수로 다시 받아야 합니다
<code>backend.tf</code> 에 버킷 이름을 또 직접 적었습니다	<code>dev</code> 와 <code>prod</code> 를 같은 코드로 만들려면 코드는 같고 <code>key</code> 만 달라야 합니다
<code>aws_subnet.public["a"]</code> 로 하나를 골랐습니다	그 서브넷이 모듈 안으로 들어가면 밖에서 어떻게 고를까요. 모듈은 안이 보이지 않습니다

참고 문서

- [Input Variables](#) : 2 · 3 · 4 · 6번
- [Custom Conditions](#) : 4 · 5 · 15번. `validation` 과 `precondition`
- [Local Values](#) : 7번
- [for_each](#) : 9 · 12번
- `count` : 10 · 11번
- [for Expressions](#) : 13번
- [terraform_data](#) : 11번의 샌드박스
- week2 개념워크북 9 · 12번 : state 주소와 교체(`-/+`)
- week3 개념워크북 4번 : `backend` 가 변수를 못 받는 이유

이 문서의 plan 출력, 오류 메시지, AZ 목록은 Terraform 1.15.8 · AWS provider 6.58.0 · 서울 리전에서 실제로 실행해 얻은 값입니다. 비용은 week3 개념워크북 Part 6에서 조회한 값을 그대로 옮겼고, 서브넷과 라우트 테이블 연결이 2개로 늘어난 부분은 둘 다 \$0 이라 합계가 같습니다. NAT Gateway 요금은 조회하지 않아 숫자를 적지 않았습니다. 원화 환산은 넣지 않았습니다.

다음 → 실습워크북 을 펴고 `practice/count-demo/` 폴더로 이동하세요.